GUIDED CAPTURE MOBILE APPLICATION

Technical Design Document

React Native application with an Android ARCore native module

Real-time capture guidance, quality analysis, session recording, metadata export and handoff to the Rysera reconstruction pipeline

Field	Detail
Prepared for	Rysera Innovations — client and technical review
Project	Transition X — guided indoor capture
Client application	React Native mobile application with an Android ARCore native module
Primary platform	Android; iOS treated as a future portability target
Target hardware	Mainstream mid-range Android phones (for example the Samsung Galaxy A-series, Oppo mid-range models), not flagship devices
Distribution	Open source; permissively licensed dependencies only
Document version	1.0
Date	27 September 2026
Status	Proposed design, issued for client and mentor review

This document supersedes the earlier architecture note. It adds the client's stated requirements on device class, open-source distribution and platform portability, completes the guidance rule set, and specifies the session state machine, threading model, calibration method and validation plan that the earlier note left open.

1. Executive summary

The application is a guided capture system for indoor spaces. While the user records a room with an ordinary phone, the application continuously receives camera and tracking frames from ARCore, converts each frame into measurable quality signals, applies a rule-based decision engine to those signals, and presents a single prioritised instruction on screen. When recording stops it summarises capture quality and produces an export package containing the recording together with frame-level metadata for downstream processing by Rysera.

The problem the application solves is behavioural rather than algorithmic. A reconstruction pipeline can only recover geometry that the camera actually observed from more than one viewpoint. An untrained user asked to scan a room will usually stand in one position and rotate, because that is the habit formed by taking panoramas. That motion produces excellent frame overlap and no parallax at all, and the reconstruction fails completely. Correcting this behaviour at the moment it occurs is more effective, and far cheaper, than detecting it after the fact and asking the user to return to the site.

The proposed architecture is hybrid. React Native implements the user interface, navigation, session summary, application state and transfer workflow. ARCore frame access, pose and point-cloud acquisition, the Recording API and all per-frame analysis remain in an Android native module written in Kotlin, exposed to React Native through a narrow command-and-event bridge. This split is not a preference: ARCore is an Android-only SDK with no JavaScript interface, and the frame loop runs at approximately camera frame rate, which is not a workload that should cross a language boundary thirty times a second on the hardware class this project targets.

Three characteristics distinguish this design from a straightforward implementation of the same idea. First, the guidance decision is exclusive and time-stabilised: several rules are true simultaneously in normal use, and showing all of them trains the user to ignore the interface. Second, the permitted message set is governed by an explicit session state machine, so the application does not emit a burst of meaningless warnings during the seconds in which ARCore is still initialising. Third, no completion percentage is displayed until the project can measure the quantity that actually matters, because the obvious way of computing coverage rewards precisely the behaviour the application exists to prevent.

2. Why guidance is required

A photogrammetric reconstruction pipeline recovers three-dimensional structure by detecting visual features in each frame, matching them across frames, estimating how the camera moved between them, and triangulating each matched feature from two or more viewpoints. Each of those steps has a precondition that an untrained operator can violate without noticing. The table below lists the failure modes the application is designed to prevent, and the guidance rule that addresses each.

Failure in the pipeline	What the user did	Rule
Consecutive frames share too little content, so feature matching breaks and the camera track is lost	Turned or walked too quickly	G1

Failure in the pipeline	What the user did	Rule
Features are observed from a single viewpoint, so triangulation has no baseline and depth cannot be recovered	Stood in one position and rotated	G4
Motion blur removes the edges that feature detection depends on	Moved while the exposure was long, or in low light	L3
Too few detectable features in view	Pointed at a blank wall, a mirror or a window	S1
Under- or over-exposed regions carry no usable detail	Scanned a dark corner, or aimed into a bright window	L1, L2
Part of the room is absent from the model	Stopped before covering the space	G3, C1, C6
The camera track is lost and the session fragments	Covered the lens, moved suddenly, or was interrupted	T1

The dominant failure mode

Rotating on the spot is the single most common untrained capture behaviour and the most damaging one. It produces high frame overlap, which makes the capture look successful to the user, while producing no parallax whatsoever, which makes it unusable to the pipeline. The rule that detects it, G4, therefore sits at the highest non-blocking priority tier and suppresses every quality message beneath it.

3. Scope boundary

The application captures and guides. It does not reconstruct. The boundary below is stated explicitly so that neither side builds the same component twice.

Within the scope of this application	Outside the scope, handled by Rysera
ARCore session lifecycle and recording control	Structure-from-Motion and feature matching
Frame, pose and point-cloud signal collection	Global camera pose optimisation and bundle adjustment
Quality metric calculation	Dense point-cloud generation
Guidance decision logic and timing	Meshing, texturing and 3D reconstruction
Live guidance interface in React Native	Floor plans and downstream products
Session logging and post-capture summary	
Export package creation and versioning	
Background and manual transfer workflow	

The interface between the two is the export package described in section 15. Everything this application produces is contained in it; everything Rysera requires should be obtainable from it without the application being consulted again.

4. Requirements

4.1 Functional requirements

ID	Requirement
FR-1	The application shall verify camera permission, ARCore availability and device support before a session can start.
FR-2	The application shall start an ARCore session and begin recording on a single user action.
FR-3	The application shall acquire, for each frame, the camera pose, tracking state, tracking failure reason and feature-point count.
FR-4	The application shall compute image quality signals on a sampled subset of frames.
FR-5	The application shall evaluate the guidance rule set against the measured signals on every frame.
FR-6	The application shall display at most one guidance message at any time.
FR-7	The application shall suppress guidance that is not meaningful in the current session state.
FR-8	The application shall display a visual record of where the user has already been.
FR-9	The application shall indicate when the capture is sufficient to stop.
FR-10	The application shall log every pose, sampled signal set and guidance event for the duration of the session.
FR-11	The application shall finalise the recording and present a session summary when the user stops.
FR-12	The application shall offer, from the summary, both acceptance of the capture and immediate re-recording.
FR-13	The application shall assemble an accepted session into a versioned export package.
FR-14	The application shall transfer an accepted package automatically when network conditions allow, and shall provide a manual transfer path otherwise.
FR-15	The application shall recover from loss of tracking without ending the session.

4.2 Non-functional requirements

These are stated as measurable targets so that they can be tested rather than asserted. The reference device is the mid-range handset agreed with the client under section 5.

ID	Requirement	Target
NFR-1	Guidance latency	A message appears within 500 ms of its condition being sustained for the debounce window
NFR-2	Capture frame rate	At least 25 FPS sustained on the reference device
NFR-3	Interface responsiveness	Start and stop controls acknowledge within 200 ms

ID	Requirement	Target
NFR-4	Thermal behaviour	A ten-minute session completes without entering severe thermal throttling
NFR-5	Storage safety	Free space is verified before recording; the session ends safely rather than failing if space is exhausted
NFR-6	Session length	A maximum duration is enforced, with a warning before the limit is reached
NFR-7	Message stability	No message is displayed for less than 1.5 seconds
NFR-8	Message density	Never more than one message on screen
NFR-9	Recoverability	Loss of tracking does not terminate the session
NFR-10	Portability	No platform-specific type appears in the decision engine
NFR-11	Licensing	Every dependency carries a permissive open-source licence
NFR-12	Format stability	The export manifest carries a format version, and format changes are recorded in a changelog
NFR-13	Determinism	Given the same signal sequence and the same clock values, the engine produces the same guidance sequence

4.3 Constraints stated by the client

Constraint	Consequence for this design
The application is built with React Native	React Native owns the interface, navigation, state and transfer workflow. ARCore access and per-frame work remain in a Kotlin native module, because ARCore has no JavaScript interface (section 6).
It must work on mainstream mid-range Android phones	Per-frame work stays native; image analysis is sampled and frame-skipped; no Phase 1 rule may depend on the ARCore Depth API; thermal and storage behaviour are explicit requirements (section 5).
It is released as open source	Apache-2.0, permissively licensed dependencies only, no committed secrets, reproducible build from a clean clone, and user-facing text externalised so that contributors can add languages without touching native code (section 21).
iOS should remain possible later	The export package is platform-neutral, and the decision engine is written without platform types so it can be ported. React Native shares the interface layer; it does not make ARCore available on iOS (section 5.5).

5. Target devices and platform portability

5.1 Device class

The application targets phones that ordinary people own, in the class of the Samsung Galaxy A-series and equivalent Oppo, Xiaomi and Motorola mid-range models, rather than flagship handsets. This is a design constraint with consequences throughout the system, not a note about testing.

Property of the device class	Consequence for the design
ARCore availability is not universal	Support is determined per model by Google, not by specification. A device-support check runs before any session begins, and an explicit unsupported-device screen is required (section 18).
The ARCore Depth API is frequently unavailable	No Phase 1 rule may depend on depth. Distance-to-surface guidance is deferred to Phase 2 and degrades to absence, not to a wrong value, when depth is missing.
Sustained CPU load causes thermal throttling sooner	Image analysis is sampled rather than run per frame, frames are dropped rather than queued under load, and a thermal rule reduces the workload before the operating system does it abruptly.
Less CPU headroom overall	The per-frame path stays in the native layer. Moving frame-rate analysis into JavaScript is explicitly rejected for this hardware class.
Storage is often close to full	Free space is checked before recording, the recording resolution is a configurable trade-off, and a maximum session length is enforced.

5.2 Device qualification procedure

Because ARCore support is granted per model and the supported list changes over time, this document does not fix a device list. It fixes the procedure and the record.

1. Agree three to four representative target models with Rysera.
2. Verify each against Google's published ARCore supported-devices list, and record the result with the date checked.
3. Install Google Play Services for AR on each and confirm a session starts.
4. Record, for each device, whether the Depth API is available, which camera configurations are offered, and the sustained frame rate over a ten-minute session.
5. Maintain the result as a device matrix in the repository, refreshed at the start of each phase.

Confirming the intended target models is an open item for Rysera; it is listed in section 23 and should be closed before implementation of the capture layer begins, because an unsupported target model changes the plan rather than the schedule.

Device	ARCore supported	Depth API	Camera configs	Sustained FPS	Date checked
(to be completed)					

5.3 Thermal, storage and battery policy

Concern	Policy
Thermal	Thermal status is read each second. On the first warning level the image-analysis sampling interval is doubled; on the next, image analysis is suspended and only pose-based rules remain active; the user is told the session should be finished soon.
Storage	Free space is checked before recording starts against an estimate derived from the selected camera configuration and the maximum session length. Recording does not begin if the estimate cannot be met.
Session length	A maximum duration is configured. The user is warned one minute before it, and the session finalises cleanly at the limit rather than being cut off.
Battery	Session length and screen brightness are the dominant costs. No background work runs during capture other than buffered logging.

5.4 Camera configuration

ARCore offers a set of camera configurations per device, and the choice affects reconstruction quality, file size and thermal load simultaneously. The selected configuration is recorded in the export manifest so that a capture can always be interpreted against the settings that produced it. The resolution and frame rate Rysera prefers is an open question (section 23); until it is answered the application selects the highest configuration that sustains the frame-rate requirement on the reference device.

5.5 Platform portability

React Native provides one implementation of the user interface across Android and iOS. It does not provide augmented reality across platforms. ARCore is an Android SDK; the iOS equivalent is ARKit, a separate Apple framework with a different API and a different language. An iOS version therefore requires a second native module written from the beginning. This is stated plainly here so that the portability benefit of React Native is neither overestimated nor dismissed.

Layer	Android	iOS, if pursued	Shared
Interface, navigation, summary, transfer	React Native	React Native	Yes
Native module and view host	Kotlin	Swift	No
AR capture and recording	ARCore	ARKit	No
Signal analysis	Kotlin	Swift	No
Decision engine	Kotlin	Swift	No, unless relocated (section 10.6)
Export package writer	Kotlin	Swift	No, but the format is identical

Two decisions are taken now rather than later, because retrofitting either after Rysera has begun consuming packages would require changes on both sides:

- **The export package is platform-neutral.** The ARCore recording format can only be read by ARCore, and ARKit has no equivalent. poses.jsonl is therefore the primary record of camera pose, and the video is an ordinary MP4. A package produced on either platform has the same structure and needs one parser.
- **The decision engine contains no platform types.** It consumes SignalFrame and produces GuidanceState using arithmetic and timers only, and it receives the current time as a parameter rather than reading a clock. It can be ported, or relocated to TypeScript, without redesign.

6. System architecture

The architecture is hybrid and layered. Each layer performs one kind of work and hands the next layer a single, well-defined object. No layer reaches past its neighbour, and no layer below the bridge knows that React Native exists.

Figure 1 — Layered architecture

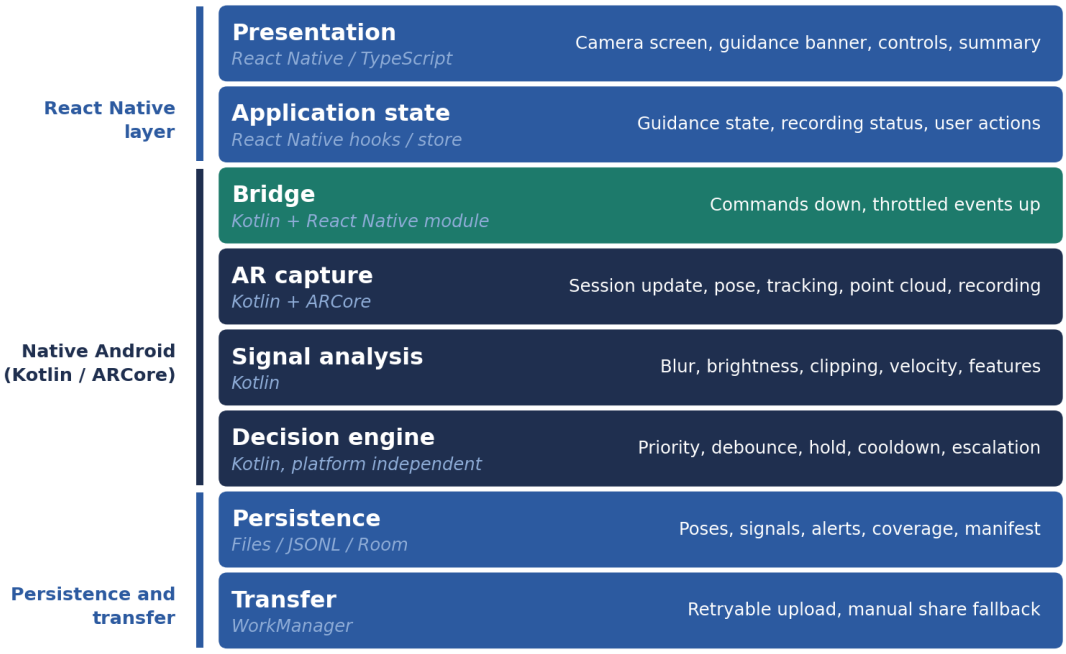


Figure 1 — Layered architecture. The React Native layer owns what the user sees and does; the native layer owns everything that runs at frame rate.

6.1 Technology assignment and rationale

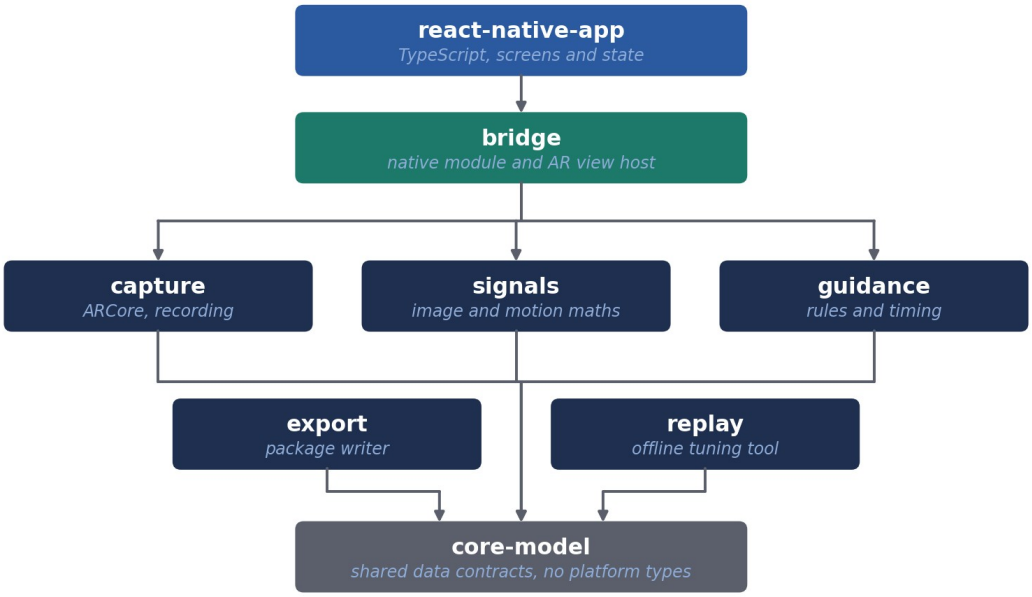
Layer	Technology	Why it belongs there
Presentation	React Native / TypeScript	Screens, navigation and styling change often and benefit most from a single cross-platform implementation.

Layer	Technology	Why it belongs there
Application state	React Native hooks or store	Session status and user actions are interface concerns, not capture concerns.
Bridge	Kotlin + React Native module	The only place the two worlds meet. Deliberately narrow: three commands down, three event types up.
AR capture	Kotlin + ARCore	ARCore is an Android SDK with no JavaScript interface. There is no alternative.
Signal analysis	Kotlin	Runs at frame rate on image buffers. Crossing a language boundary per frame is the wrong trade on mid-range hardware.
Decision engine	Kotlin, platform independent	Must be deterministic and testable without a device. Written with no Android or ARCore types (see section 10.6).
Persistence	Files, JSONL, optionally Room	Append-only logs suit frame-rate writing; a database is only needed if on-device querying is required.
Transfer	Android WorkManager	Retryable background upload under network constraints, with a manual fallback.

6.2 Module structure

The native side is divided into modules with a strictly downward dependency direction. The value of this is practical rather than aesthetic: core-model, signals and guidance carry no Android dependency, which means they compile and run as plain Kotlin and can be tested on a workstation with no device attached. That is what makes the offline calibration described in section 19 possible.

Figure 2 — Module structure and dependency direction



Arrows point from a module to the module it depends on. No arrow ever points upward.

Figure 2 — Module structure. core-model holds the shared data contracts; signals, guidance and replay contain no platform types.

Module	Contents	Platform dependency
core-model	The three data contracts and their supporting types	None
capture	ARCore session, pose, point cloud, Recording API	ARCore
signals	Image and motion measurement	None
guidance	Rules, priority, timing, state machine	None
export	Package writer, manifest, logs	Android file system
replay	Offline harness for threshold sweeps	None
bridge	React Native module, event emitter, AR view host	Android + React Native
react-native-app	Screens, state, strings, transfer workflow	React Native

7. Data contracts

Three objects define every boundary in the system. They are declared in core-model, contain no behaviour, and carry no Android or ARCore types. Fixing them early is what allows the capture, analysis, decision and interface work to proceed in parallel; changing one later affects several modules at once, so they are treated as a stable interface rather than as convenience structures.

Figure 3 — The capture loop, repeated about thirty times a second

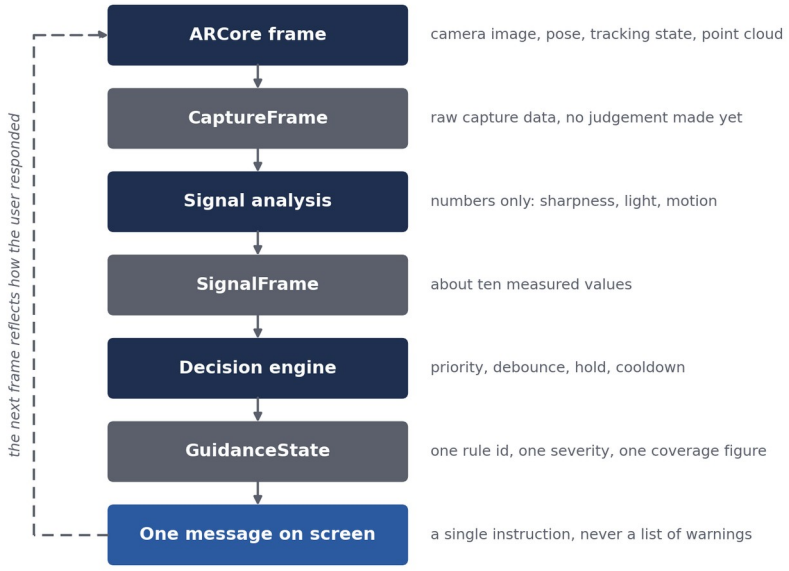


Figure 3 — The capture loop. Each stage reduces the data: an image becomes ten numbers, ten numbers become one instruction.

7.1 CaptureFrame — capture layer to analysis layer

Everything the capture layer extracted from one ARCore frame. It contains no judgement about whether the frame is good. Because ARCore types cannot appear in core-model, the ARCore pose, tracking state and point cloud are represented by plain equivalents and mapped at the capture-layer boundary.

```
data class CapturePose(
    val tx: Float, val ty: Float, val tz: Float,
    val qx: Float, val qy: Float, val qz: Float, val qw: Float
) {
    fun translationDistanceTo(other: CapturePose): Float
    fun angleDegreesTo(other: CapturePose): Float
}

enum class TrackingStatus  { TRACKING, PAUSED, STOPPED }
enum class TrackingFailure { NONE, EXCESSIVE_MOTION, INSUFFICIENT_FEATURES,
                             INSUFFICIENT_LIGHT, BAD_STATE, CAMERA_UNAVAILABLE }

data class CaptureFrame(
    val timestampNs: Long,
    val pose: CapturePose,
    val trackingStatus: TrackingStatus,
    val trackingFailure: TrackingFailure,
    val lumaPlane: ByteArray?,        // null when this frame is not sampled
    val width: Int, val height: Int,
    val featurePoints: FloatArray?,
    val featureCount: Int
)
```

The image is carried as the luminance plane of the YUV buffer rather than as an RGB bitmap. Every image measurement in this design operates on brightness alone, so colour conversion would be pure cost. The Y plane is already the correctly weighted luminance; computing it as a mean of the three colour channels would be both slower and wrong.

7.2 SignalFrame — analysis layer to decision engine

The measurement result. Numbers only; no rule is evaluated here and no message is selected. Image-derived fields are refreshed on sampled frames and carried forward between samples, so that every SignalFrame is complete.

```
data class SignalFrame(
    val timestampNs: Long,
    val blurScore: Float,            // Laplacian variance; higher is sharper
    val brightness: Float,           // mean luminance, 0..255
    val clippedHigh: Float,          // percentage of pixels above 250
    val clippedLow: Float,           // percentage of pixels below 5
    val angularVelocity: Float,      // degrees per second
    val linearVelocity: Float,       // metres per second
    val featureCount: Int,
    val baseline: Float,             // metres travelled since the last keyframe
    val distanceToSurface: Float?,   // Phase 2; null when depth is unavailable
    val trackingOk: Boolean,
    val failureReason: TrackingFailure,
    val thermalStatus: Int
)
```

7.3 GuidanceState — decision engine to interface

What the interface should display now. It carries a rule identifier rather than a rendered sentence; the reasoning is given in section 11.2.

```
enum class CoverageMethod { NONE, POSE_TRAIL, POINTCLOUD_ANGULAR, VOXEL_GRID }

data class CoverageState(
    val method: CoverageMethod,
    val surfacesSeenFromTwoPlusAngles: Float,   // 0..1, the meaningful figure
    val observedVoxelCount: Int,
    val coveredVoxelCount: Int
)

data class GuidanceState(
    val ruleId: GuidanceRuleId?,     // null when nothing should be shown
    val severity: Severity,          // INFO, WARNING, CRITICAL
    val sessionState: SessionState,
    val coverage: CoverageState,
    val canStop: Boolean,
    val escalated: Boolean,
    val elapsedMs: Long
)
```

Why coverage is an object and not a percentage

The straightforward way to compute coverage is to divide the sphere of viewing directions into sectors and count how many have been observed. That measure reports its maximum for a user standing still and rotating, which is the worst capture the pipeline can receive, and a low value for a user walking correctly around the room. It is not merely imprecise; it is inverted with respect to what matters.

CoverageState therefore names what was measured and how. For the first release the method is POSE_TRAIL and the interface draws the path the user has walked instead of a percentage. A percentage is introduced only once surfacesSeenFromTwoPlusAngles can be computed, because that is the quantity the reconstruction actually depends on. The method is written into the export manifest so that a capture can always be interpreted against the definition that produced it.

8. Layer 1 — AR capture

The capture layer owns the ARCore session. On each call to session.update() it receives a Frame and extracts the data the rest of the system needs, packaging it as a CaptureFrame. It makes no judgement about quality.

Acquired from ARCore	Used for
camera.pose	Angular and linear velocity, baseline, the pose trail, and every pose written to poses.jsonl
trackingState	The session state machine and rule T1
trackingFailureReason	Rules T4, T5 and T6, with no thresholds required
acquirePointCloud()	Feature count for rule S1

Acquired from ARCore	Used for
Camera image, Y plane only, on sampled frames	Blur, brightness and clipping measurement
Frame timestamp	Correlating every log stream and the recording
Camera intrinsics	The export manifest, to assist the downstream pipeline

Recording is controlled through the ARCore Recording API and is deliberately kept separate from guidance evaluation. Starting and stopping a recording briefly pauses the ARCore session; guidance evaluation is suspended across those transitions so that the pause is not reported to the user as a loss of tracking. Custom track data may be muxed into the recording, but at a controlled rate rather than on every frame.

Technical risk carried by this layer

The combination of SharedCamera, the Recording API and playback is the least certain part of the design and cannot be validated on an emulator. It is scheduled first, and a fallback exists: record video and pose data as separate synchronised streams rather than as a single ARCore recording. Because the export package is already specified as platform-neutral (section 15), that fallback changes the capture implementation without changing what Rysera receives.

9. Layer 2 — Signal analysis

The analysis layer converts images and poses into numbers. Every function is pure: given the same input it returns the same output, holds no state beyond the previous pose, and touches no Android type. That is what allows it to be tested against synthetic images on a workstation, with no camera involved.

Signal	Method	Detects
blurScore	Downsample the luminance plane to 160 × 120, apply a 3 × 3 Laplacian kernel, take the variance of the result. Higher is sharper.	Motion blur, focus failure
brightness	Mean of the luminance plane	Under-exposure
clippedHigh / clippedLow	Percentage of pixels above 250 and below 5	Blown highlights, crushed shadows
angularVelocity	Rotation between consecutive poses, divided by the frame interval	Turning too fast
linearVelocity	Translation between consecutive poses, divided by the frame interval	Walking too fast, or not at all
featureCount	Size of the current ARCore point cloud	Blank walls, mirrors, windows
baseline	Distance travelled since the last accepted keyframe	Standing still

Figure 6 — What each signal is actually measuring

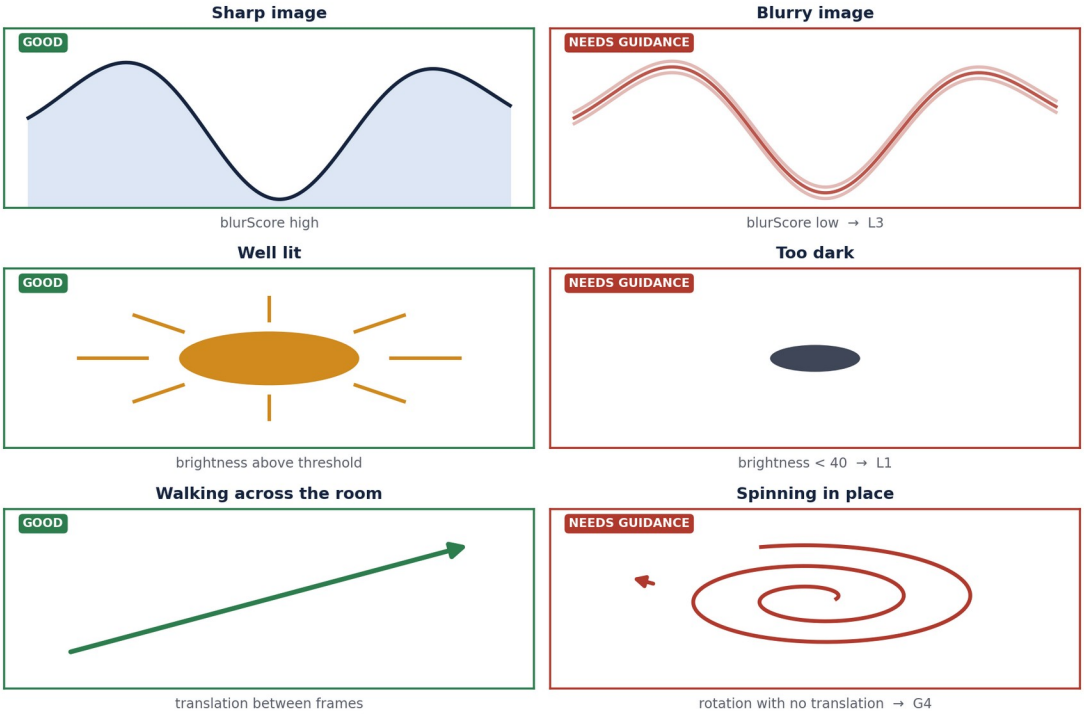


Figure 6 — What each signal measures. The right-hand column shows the condition each rule is designed to catch.

9.1 Frame budget

Not every measurement needs to run at frame rate. Pose arithmetic is cheap and time-sensitive; image processing is comparatively expensive and changes slowly.

Work	Frequency	Reason
Pose deltas, tracking state	Every frame	Low cost, and the rules that use them must react immediately
Feature count	Every frame, throttled if required	Provided by ARCore at negligible cost
Blur, brightness, clipping	Every third to fifth frame	Heaviest work in the loop; the underlying conditions change over hundreds of milliseconds, not tens
Emission to React Native	On meaningful change only	Avoids bridge traffic that no one reads
Disk logging	Buffered and batched	Per-frame file I/O would dominate the budget

10. Layer 3 — Decision engine

This is the layer that determines whether the application is useful or merely noisy. Converting a set of measurements into a warning is straightforward. Deciding which single warning to show, when to show it, how long to keep it, and when to say nothing at all is the part that requires design.

10.1 Rule catalogue

Phase 1 is the set below, and is the scope of the first working application. Rules marked Phase 2 are defined now so that identifiers are stable, and implemented once depth and coverage measurement are available.

ID	Condition	Tier	Guidance to the user	Phase
T1	trackingStatus is PAUSED	0	Tracking lost — move back to an area you have already scanned	1
T2	Session is initialising	0	Move the phone slowly from side to side	1
T4	failureReason CAMERA_UNAVAILABLE	0	Another app is using the camera — close it and try again	1
T5	failureReason BAD_STATE	0	Something went wrong — please restart the scan	1
G4	High rotation with near-zero translation, sustained	1	Walk around the room, don't spin in place	1
G1	angularVelocity above threshold	2	Turn more slowly	1
L1	brightness below threshold	2	It's too dark here — turn on a light	1
L3	blurScore below threshold	2	The image is blurry — move more slowly	1
S1	featureCount below threshold	2	Point at a corner or an object, not a blank wall	1
G2	Pose jitter above threshold	3	Hold the phone steady	1
L2	clippedHigh above threshold	3	Too bright — avoid pointing at the window	1
T6	failureReason EXCESSIVE_MOTION	3	You're moving too fast	1
G3	baseline below threshold, sustained	5	Keep moving forward	1
C6	Coverage target reached	7	This scan looks good — you can stop	1
L4, L5	Lighting changes, flicker	4	—	2
S2, S3	Too close, too far (requires depth)	3, 4	—	2
T3	Thermal throttling	4	—	2
C1 – C3, C5	Directional and completeness guidance	5, 6	—	2
S4, C4	Mirrors, corner handling	—	—	3

Rules T1, T2, T4, T5 and T6 derive directly from ARCore's own tracking state and failure reason. They require no thresholds and no calibration, and they are the most reliable signals available in the whole system. The earlier architecture note collected this information but defined no rule that used it; that gap is closed here.

10.2 Priority tiers

Several rules are true at once in ordinary use. The engine selects exactly one. The selection is by tier, and the tier expresses how much of the capture is at stake, not how alarming the condition sounds.

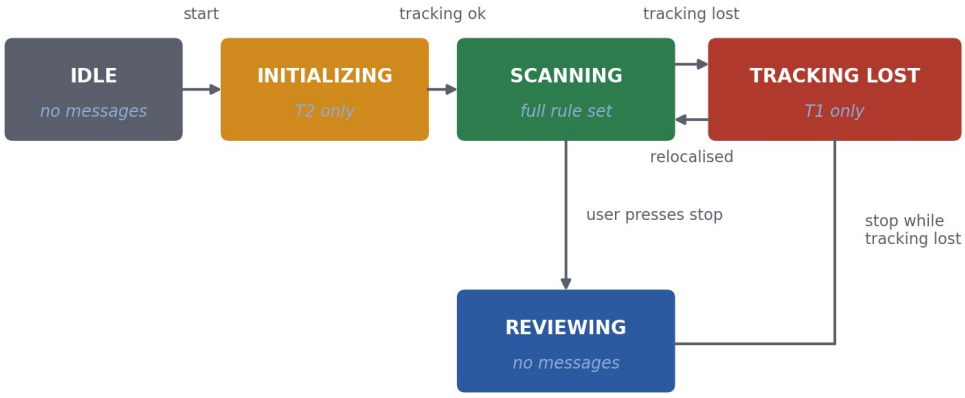
Tier	Meaning	Rules
0	Session blocking. Nothing else is actionable while this holds.	T1, T2, T4, T5
1	The capture is structurally unusable, however good it looks.	G4
2	Data is being actively destroyed.	G1, L1, L3, S1
3	Data quality is reduced.	G2, L2, S2, T6
4	Efficiency and device health.	L4, L5, S3, T3
5	Completeness.	G3, C3, C5
6	Navigation.	C1, C2, C4
7	Positive confirmation.	C6

Severity is derived from the tier of the selected rule; it exists so the interface can choose a colour and decide whether to use haptic feedback. The tier is the single source of truth, and no rule carries an independent severity of its own.

10.3 Session state machine

Rule evaluation is not meaningful at all times. While ARCore is initialising the pose estimate is unreliable, the feature count is low and image measurements are meaningless. Evaluating the normal rule set during that window produces a burst of warnings in the first seconds of every session — which is the user's first impression of the application.

Figure 4 — Session state machine and the messages each state permits



The permitted message set is a property of the state, enforced by the state machine — not a condition written inside each rule.

Figure 4 — Session states and the messages each permits.

The permitted message set is a property of the state, enforced by the state machine's own transition table. It is deliberately not implemented as a condition inside each rule, because a rule added a year from now will not remember to include the check. Entry into SCANNING additionally requires tracking to have been stable for a short settling period, so that a momentary recovery does not immediately re-enable the full rule set.

10.4 Temporal stabilisation

Raw rule output is unusable directly. Signals fluctuate frame to frame, and a message that appears and disappears five times a second is worse than no message.

Mechanism	Behaviour	Prevents
Hysteresis	Each rule has separate entry and exit thresholds, for example enter below 100 and exit above 150	Flicker when a signal sits on the threshold
Debounce	A rule must hold for three consecutive frames before it can be selected	Reaction to a single noisy frame
Minimum display time	Once shown, a message stays for at least 1.5 seconds	Messages disappearing before they can be read
Cooldown	The same message appears at most three times in ten seconds	Repetition that trains the user to ignore the banner
Escalation	On the third occurrence the state is marked escalated so the interface can show a fuller explanation	A short instruction repeating when the user has not understood it
Settling period	Guidance is suppressed briefly after entering SCANNING and across recording transitions	Warnings caused by the system rather than by the user

The engine receives the current time as a parameter and never reads a system clock internally. This is required twice over: it is what makes the timing behaviour testable without waiting in real time, and it is what allows a recorded session to be replayed faster than real time during calibration (section 19).

10.5 Worked example

Figure 5 — Three rules are true at once; the user sees one message

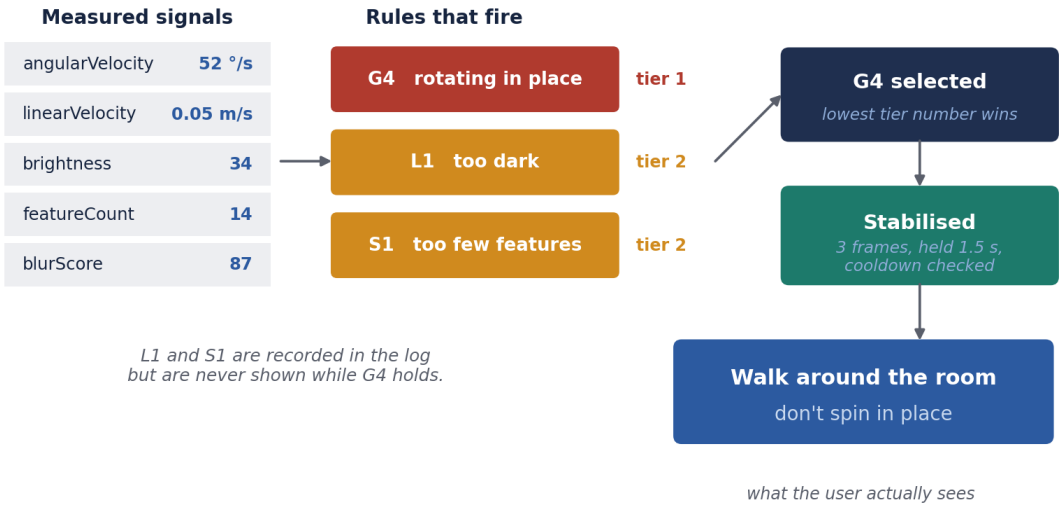


Figure 5 — Three rules fire; one instruction reaches the user. The suppressed rules are still written to the alert log.

10.6 Where the decision engine should live

ARCore access and signal analysis must be native; there is no choice. The decision engine is the one component where a genuine choice exists, because it performs only arithmetic and timing on about ten numbers per frame. It could equally run in TypeScript. This section sets out both options and makes a recommendation.

Consideration	Kotlin, in the native module	TypeScript, in React Native
Bridge traffic	None. The engine sits beside the analyser.	About ten numbers per frame. Small in absolute terms, but continuous.
Timing accuracy	Runs on a dedicated thread; debounce and cooldown timers are unaffected by interface work.	Shares the JavaScript thread with rendering. A re-render or a garbage-collection pause delays timers.
Offline replay	The replay harness runs the real engine on a workstation with no device.	A separate Node harness would have to be built and kept in step.
Tuning cycle	Thresholds load from a configuration file, so tuning does not require a rebuild; changing rule logic does.	Rule logic can be changed with hot reload, which is faster during exploratory tuning.
Contribution barrier	Contributors need a working Android toolchain.	Contributors need only the JavaScript toolchain.
Reuse on iOS	Requires a port to Swift.	Shared unchanged.
Mid-range hardware	No additional JavaScript-thread load during capture.	Adds continuous work to the thread that also drives the interface.

Recommendation

Implement the decision engine in Kotlin for this project, and preserve the option to relocate it.

The determining factor is not performance — the arithmetic is trivial either way — but the offline replay harness described in section 19. Calibrating thresholds against labelled captures is the step that turns the rule set from a set of guesses into a defensible result, and it depends on running the real engine, at many times real speed, against recorded sessions with no device attached. In Kotlin that harness is the same code as the application. In TypeScript it would be a second implementation, and a second implementation that drifts is worse than no harness at all. Timing determinism on mid-range hardware is a secondary but real argument in the same direction.

The option is preserved at no cost: the engine takes no platform types and receives time as a parameter, and the bridge emits SignalFrame as well as GuidanceState, so its input is already available in JavaScript. If Rysera later commits to iOS, or prefers rules to be editable by web developers, the engine can be transcribed rather than redesigned.

This remains a decision for the client and mentor to confirm; it is listed in section 23.

11. Layer 4 — React Native interface

The interface subscribes to guidance events emitted by the native module and renders the current state. It is intentionally thin: it does not re-derive anything the engine has already decided, and it never evaluates a threshold.

Figure 7 — The two screens the user moves between

One message at a time. Re-recording is offered immediately, while the user is still in the room.

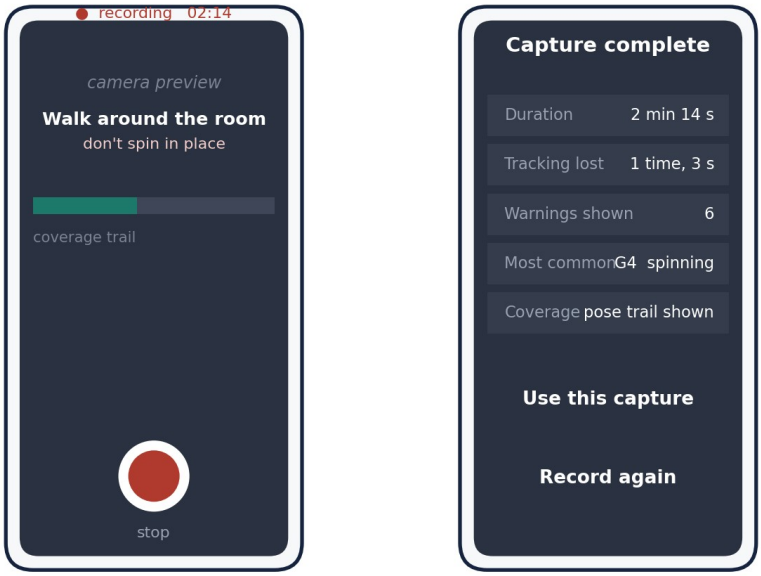


Figure 7 — The capture screen and the session summary.

11.1 Screen elements

Element	Behaviour
Camera and AR view	A native view hosted inside the React Native screen. ARCore renders to a surface that JavaScript does not touch.
Guidance banner	One message at a time. Colour and haptic feedback follow severity. The message text is resolved from a string resource using the rule identifier.
Pose trail	A visual record of where the user has walked. In the first release this replaces a coverage percentage.
Status line	Recording indicator, elapsed time, tracking state.
Start and stop controls	A single large control; stop is always available.
Summary screen	Duration, tracking losses, warnings by rule, coverage, and two actions: use this capture, or record again.

Offering immediate re-recording matters more than it appears to. The user is still standing in the room at the moment the summary is shown. A capture rejected then costs two minutes; the same capture rejected by the pipeline the following day costs another journey.

11.2 Guidance text and localisation

GuidanceState carries a rule identifier, not a rendered sentence. The interface resolves the text for the current rule from a React Native string resource. There are three reasons, and the first is a consequence of the open-source requirement.

6. Adding Sinhala, Tamil or any other language must not require rebuilding the native module. If the message text is compiled into Kotlin, a translation contribution becomes an Android toolchain task.
7. The interface can style, animate and prioritise per rule, which it cannot do from an opaque string.
8. Message wording will change repeatedly during user testing. Wording changes should not be native code changes.

A plain-text message field may be carried in development builds for debugging, but it is never the source of displayed text.

12. React Native to native interface

The bridge is deliberately narrow: session-level commands downward, low-bandwidth state upward. Full image buffers never cross it.

Direction	Interface	Purpose
RN → native	startCapture(config)	Open the ARCore session, begin recording, start analysis
RN → native	stopCapture()	Stop recording, finalise metadata, compute the summary

Direction	Interface	Purpose
RN → native	acceptSession()	Mark the session for packaging and transfer
RN → native	discardSession()	Delete the recording and logs for a rejected capture
native → RN	guidanceChanged	GuidanceState, emitted only when the selected rule or severity changes
native → RN	captureStatus	Tracking state, elapsed time, coverage, errors; emitted at a controlled interval
native → RN	signalFrame	SignalFrame, throttled to roughly 5 Hz, for the debug overlay
native → RN	sessionCompleted	Summary and the local package location

The signalFrame event is included for three reasons that justify its small cost: it is the only way to build an on-device debug overlay; it allows a tester to see why a message appeared rather than only that it did; and it keeps the relocation option in section 10.6 open without further work.

Implementation note on the React Native native module

React Native's native module and native component APIs changed substantially with the New Architecture, and a large amount of published tutorial material still describes the superseded approach. The AR view in particular is a native view component, which is significantly more involved than exposing a method. The bridge should therefore be written against the current architecture from the start, and the React Native version should be pinned in the repository. This is the least familiar part of the work and is scheduled accordingly.

13. Threading and performance

Thread ownership is specified rather than left to emerge, because on the target hardware it is the difference between meeting and missing the frame-rate requirement.

Figure 8 — Thread ownership and backpressure

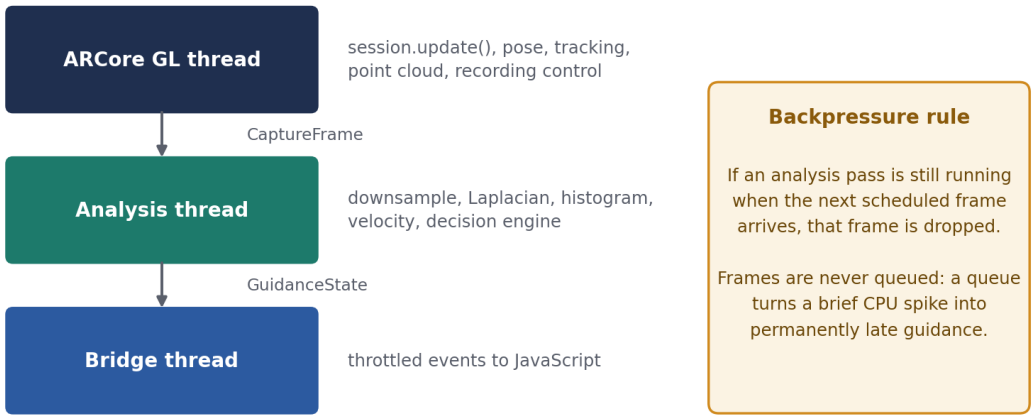


Image analysis never runs on the GL thread — doing so stalls session.update() and drops the capture frame rate.

Figure 8 — Thread ownership and the backpressure rule.

Work	Thread
session.update(), pose, tracking state, point cloud, recording control	ARCore GL thread
Downsampling, Laplacian, histogram, velocity	Analysis thread
Decision engine	Analysis thread, single-threaded and deterministic
Event emission to JavaScript	Marshalled to the bridge thread
Log buffering and disk writes	Background writer thread
Interface rendering	React Native JavaScript and UI threads

- Image analysis never runs on the GL thread. Doing so stalls session.update() and drops the capture frame rate directly.
- If an analysis pass has not completed when the next scheduled frame arrives, that frame is dropped. Frames are never queued: a queue converts a momentary CPU spike into permanently late guidance, which is worse than a missed sample.
- Guidance evaluation is suspended across recording start and stop, because the ARCore session pauses briefly at those points.

14. Recording, logging and session summary

The recording is the primary artefact. Logging runs alongside it and is kept logically separate from recording control, so that a change to one does not disturb the other.

Stream	Written	Contents
Recording	Continuously by ARCore	Video and AR session data
poses.jsonl	Every frame, buffered	Timestamp, pose, tracking state
quality.jsonl	Every sampled frame, buffered	The full SignalFrame
alerts.jsonl	On every guidance change	Rule identifier, severity, time shown, time dismissed, escalation
coverage.json	At finalisation	Final CoverageState including the method used

These logs exist so that the question of whether guidance changed behaviour can be answered from data rather than from impression. Specifically they support: how often each rule fired; how long each message was visible; whether linear velocity increased in the seconds following a G4 warning; how often tracking was lost; and what coverage was reached.

When the user stops, the native layer finalises the recording, flushes the log buffers, computes the summary and returns it to React Native.

Summary metric	Purpose
Total recording time	Session duration
Coverage, with method stated	Indication of completeness
Tracking losses: count and total duration	Capture stability
Warnings by rule	Quality and usability diagnostics
Final capture status	Supports the accept or re-record decision

15. Export package

An accepted session is assembled into a single directory. This directory is the entire interface to Rysera.

```
scan_<uuid>/
  session.mp4        video of the capture
  poses.jsonl        per-frame camera pose  (primary pose record)
  quality.jsonl      per-sample signal values
  alerts.jsonl       guidance events shown to the user
  coverage.json      final coverage, including the method used
  manifest.json      device, camera intrinsics, versions
```

poses.jsonl is the primary pose record rather than a convenience copy. An ARCore recording can only be read back by ARCore, and ARKit has no equivalent format, so a package that depends on it is tied to one platform and one SDK version. With poses carried explicitly, the package has the same structure whatever produced it and Rysera needs one parser.

manifest.json

```
{
  "formatVersion": "1.0",
  "sessionId": "<uuid>",
  "device":   { "manufacturer": "...", "model": "...", "osVersion": "..." },
```

```
  "camera":   { "imageWidth": 0, "imageHeight": 0, "fps": 30,
                "fx": 0.0, "fy": 0.0, "cx": 0.0, "cy": 0.0 },
  "coverage": { "method": "POSE_TRAIL" },
  "app":      { "version": "...", "thresholdsVersion": "..." },
  "createdAt": "..."
}
```

- Camera intrinsics are included because supplying them reduces the pipeline's dependence on self-calibration and improves both the speed and the stability of reconstruction.
- formatVersion is included so that a later change to the package does not silently break a parser Rysera has already written. Format changes are recorded in a changelog kept alongside the specification.
- thresholdsVersion records which calibrated threshold set produced the guidance in this session, so that captures can be compared meaningfully across the project.

16. Transfer

Background transfer is implemented with Android WorkManager, which retries when network constraints are satisfied rather than failing at the moment of capture. Captures are frequently made in buildings with poor connectivity, so transfer is assumed to be deferred by default rather than immediate.

A manual path remains available and is specified rather than left open: the package directory is archived and offered through the Android share sheet, which allows the operator to place it in cloud storage, send it by email or copy it over a cable. The manual path is retained permanently, not only until an endpoint exists, because it is the fallback when an endpoint is unreachable. The upload endpoint and any authentication scheme are an open question for Rysera (section 23), and no endpoint or credential is committed to the repository.

17. Runtime sequence

Figure 9 — Runtime sequence, from start to accepted export

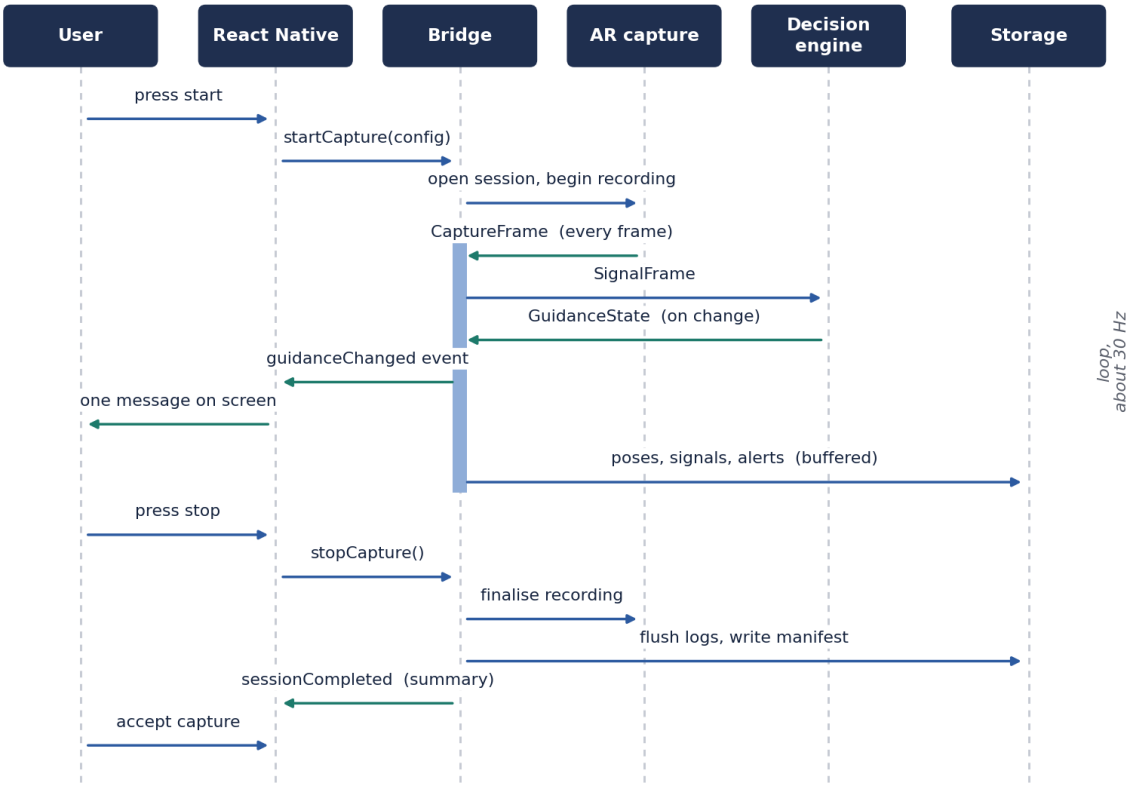


Figure 9 — Runtime sequence from session start to accepted export.

18. Error, permission and unsupported-device flows

On the target hardware an unsupported device is an expected path rather than an edge case, so these flows are specified as part of the design.

Condition	Behaviour
Camera permission denied	Explain what the permission is for and offer to request it again; do not show a blank camera screen.
Google Play Services for AR not installed	Offer the installation flow and retry the support check once it completes.
Device not supported by ARCore	State plainly that this phone cannot run the scan, and stop. Do not attempt a degraded capture.
Insufficient free storage	Report the space required before recording starts; do not begin and fail partway.

Condition	Behaviour
Recording fails during a session	Preserve whatever has been written, finalise the logs and inform the user that the capture is incomplete.
Session interrupted by a call or by the app moving to the background	Pause cleanly, attempt to resume on return, and record the interruption in the session log.
Tracking lost	Remain in the session, show T1 only, and guide the user back to a previously scanned area.
Thermal limit reached	Reduce analysis, then suspend image analysis, then advise the user to end the session.

19. Threshold calibration method

The threshold values in this document are starting points for controlled testing, not calibrated constants. Publishing a number without stating how it will be validated is the difference between a design and a guess, so the method is given here.

Signal	Initial value	Status
angularVelocity	45 °/s	To be calibrated
linearVelocity	1.0 m/s	To be calibrated
blurScore (Laplacian variance)	100	Device dependent; to be calibrated per device class
brightness	below 40 dark, above 200 bright	To be calibrated
clipping	15 %	To be calibrated
featureCount	20	To be calibrated
baseline	0.15 m	To be calibrated
G4 spin detection	30° rotation with under 0.1 m translation, over 2 s	To be calibrated

Thresholds are held in a versioned configuration file rather than in source, so that a calibration result is data rather than a code change, and so that two sets can be compared directly.

9. Record twenty to thirty capture sessions across varied rooms, lighting conditions and user behaviour, including captures made deliberately badly.
10. Submit them to the Rysera pipeline and obtain a pass or fail result for each. This labelled set is the ground truth, and obtaining it is a dependency on the client.
11. Replay the recorded sessions through the real decision engine offline, faster than real time, under swept threshold values.

12. Select thresholds by precision and recall against the labels. A missed bad capture wastes the operator's journey; a false warning teaches the operator to ignore the banner. The two errors are not equally costly and the balance is a decision to be made with Rysera, not a default.
13. Commit each resulting set as a versioned file, and record its identifier in every manifest produced under it.

Dependency on Rysera

Step 2 cannot be performed by the project alone. Without pass or fail labels from the reconstruction pipeline, threshold selection remains guesswork and the guidance cannot be shown to be correct rather than merely plausible. This is the single most valuable input the client can provide, and it is on the critical path: it is needed roughly at the midpoint of the schedule, not at the end.

20. Validation plan

Building the application demonstrates feasibility. Demonstrating that the guidance changes what users do, and that the change improves reconstruction outcomes, is the result the project is for. The two are not the same claim and the second requires a study.

Element	Design
Participants	Ten to fifteen people with no 3D reconstruction background
Conditions	A: capture with no guidance. B: capture with guidance.
Design	Within subjects, order counterbalanced, comparable rooms
Primary measure	Reconstruction success rate, judged by the Rysera pipeline
Secondary measures	Proportion of frames registered, capture duration, tracking losses, usability score
Behavioural measure	Change in linear velocity and in the rotation-to-translation ratio in the seconds following a G4 warning
Build control	All participants use one frozen build with one threshold set; any fix during the study is branched and recorded

The behavioural measure is worth noting separately. The alert log makes it possible to ask which specific messages changed behaviour and which were ignored. A finding that four of fourteen rules do the work is a useful result for the design of any future guided capture system, and it is obtainable from data the application already records.

21. Open source and licensing

The application is released as open source. This is a design constraint, not only a publication decision, and several choices elsewhere in this document follow from it.

Item	Decision
Licence	Apache-2.0, preferred over MIT for its explicit patent grant, which is worth having in a project built against vendor AR SDKs
Dependencies	Permissive licences only: Apache-2.0, MIT, BSD. No paid, trial or source-available component may enter the dependency tree.
Vendor runtime	Google Play Services for AR is a device-side runtime dependency, not a bundled component. This is compatible with an open-source client.
Secrets	No API keys, endpoints or credentials are committed. Any Rysera endpoint is supplied at build or run time through configuration.
Reproducibility	The repository builds from a clean clone using documented steps only, with the React Native and Gradle versions pinned.
Documentation	README, CONTRIBUTING, a licence header policy, and a documented build for both the JavaScript and Android toolchains.
Contribution surface	Guidance text lives in React Native string resources, so a translation or a wording improvement does not require an Android toolchain.
Licence audit	Dependency licences are checked in continuous integration, so a non-permissive transitive dependency fails the build rather than being discovered at release.

22. Risks and mitigations

Risk	Impact	Mitigation
SharedCamera, the Recording API and playback do not work together as required	High — changes the capture design	Validated first, before dependent work begins. Fallback: record video and poses as separate synchronised streams. The export format is unaffected.
Target devices are not on the ARCore supported list	High — changes the target, not the schedule	Device qualification (section 5.2) completed before capture implementation begins.
Labelled ground truth is not available from Rysera	High — thresholds remain uncalibrated	Requested early, with a stated date. Fallback: internal labelling by visual inspection, reported as a weaker result rather than presented as equivalent.
React Native native view hosting proves harder than estimated	Medium — delays the visible interface	Built against the current architecture from the start, with the version pinned. A fake data source allows the interface to be developed before the AR view exists.
Thermal throttling on mid-range devices limits session length	Medium — shorter usable sessions	Frame skipping, drop-not-queue backpressure, a thermal policy, and an enforced maximum session length.
Guidance is accurate but users ignore it	Medium — the application works and does not help	Exclusive priority, temporal stabilisation and escalation. The user study measures this directly rather than assuming it.
Export format changes after Rysera has integrated	Medium — rework on both sides	formatVersion from the first release, a changelog, and a platform-neutral package agreed before integration begins.

23. Open questions for Rysera and the mentor

These are the decisions the project cannot make alone. They are ordered by how much they affect the work, and the first three are on the critical path.

#	Question	Why it matters
1	Can Rysera provide pass or fail labels for twenty to thirty test captures, and by what date?	Without labelled ground truth the thresholds cannot be calibrated and the guidance cannot be shown to be correct (section 19).
2	Which phone models should be treated as the reference devices?	ARCore support is granted per model. An unsupported reference device changes the plan rather than the schedule (section 5.2).
3	What resolution, frame rate and codec does the pipeline prefer, and does compression degrade reconstruction?	Determines the camera configuration, which affects quality, file size and thermal load simultaneously (section 5.4).
4	Should the decision engine remain in Kotlin, or should only signals be native with the rules in TypeScript?	The one genuinely open architectural choice. A recommendation is given in section 10.6.
5	Are ARCore poses useful to the pipeline as initialisation, or will it estimate poses independently?	Determines whether pose accuracy is a deliverable or a convenience.
6	Are camera intrinsics required once per session, per camera configuration, or per frame?	Determines the manifest structure (section 15).
7	What quality criteria does the pipeline already apply — distance range, overlap, blur tolerance?	Any existing numbers are better starting points than the estimates in section 19.
8	Which capture mistakes do current users make most often?	Confirms or corrects the priority ordering in section 10.2.
9	What definition of coverage should the first release use?	The interface shows a pose trail until this is settled (section 7.3).
10	What upload endpoint and authentication should the application use?	Until this is answered, transfer is manual by design and no endpoint is committed (section 16).
11	Should frame-level logs be JSONL only, or also stored in Room for on-device evaluation?	JSONL alone is assumed unless on-device querying is required.
12	Is iOS a committed goal or a possibility?	Determines whether portability is a design constraint to invest in or an option to preserve at low cost (section 5.5).

Appendix A — Object-oriented analysis and design

This appendix presents the same system in the notation required for the object-oriented analysis and design deliverable. It adds no new design decisions; it restates the design above as use cases, an activity model, a class model and an allocation of implementation responsibility.

A.1 Use case diagram

Figure 10 — Use case diagram

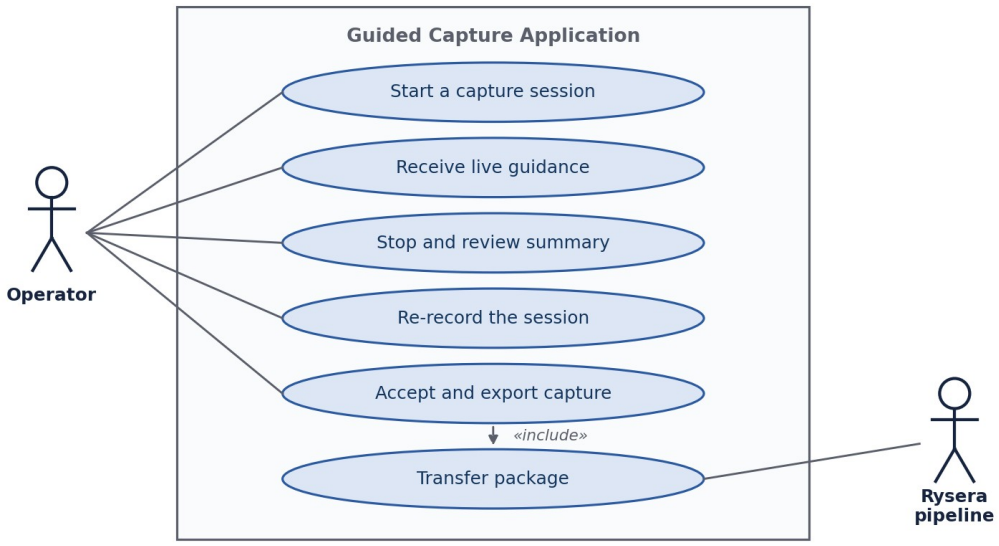


Figure 10 — Use case diagram. The operator is the person capturing the room; the Rysera pipeline is a secondary actor that receives the exported package.

A.2 Use case descriptions

UC-1 Start a capture session

Actor	Operator
Precondition	Camera permission granted; ARCore available; the device is supported; sufficient free storage
Main flow	1. The operator opens the capture screen. 2. The application verifies permissions, ARCore availability and free space. 3. The operator presses start. 4. The application opens an ARCore session and begins recording. 5. The application enters INITIALIZING and shows T2 only. 6. When tracking is stable the application enters SCANNING.
Alternative flows	The device is unsupported (UC-1a): the application explains and stops. Permission is denied (UC-1b): the application explains and offers to request again. Storage is insufficient (UC-1c): the application reports the space required and does not start.

UC-2 Receive live guidance

Actor	Operator
Precondition	A session is in SCANNING
Main flow	1. For each frame the application builds a CaptureFrame. 2. On sampled frames it computes a SignalFrame. 3. The decision engine evaluates the rule set. 4. It selects the lowest-tier rule that holds. 5. It applies debounce, minimum display time and cooldown. 6. If the selection changed, the interface shows the new message.
Alternative flows	Tracking is lost (UC-2a): the application enters TRACKING_LOST and shows T1 only until relocalisation.

UC-3 Stop and review the summary

Actor	Operator
Precondition	A session is active
Main flow	1. The operator presses stop. 2. The application stops recording and finalises it. 3. Log buffers are flushed and coverage is computed. 4. The summary is returned to the interface and displayed.
Alternative flows	Recording failed during the session (UC-3a): partial data is preserved and the operator is told the capture is incomplete.

UC-4 Re-record the session

Actor	Operator
Precondition	The summary is displayed
Main flow	1. The operator chooses to record again. 2. The application discards the rejected recording and logs. 3. UC-1 begins from step 3.
Alternative flows	—

UC-5 Accept and export the capture

Actor	Operator
Precondition	The summary is displayed
Main flow	1. The operator accepts the capture. 2. The application assembles the export package. 3. The manifest is written with intrinsics, versions and the coverage method. 4. The package is queued for transfer (includes UC-6).

Alternative flows	Storage is insufficient for packaging (UC-5a): the operator is asked to free space and retry.

UC-6 Transfer the package

Actor	Operator, Rysera pipeline
Precondition	An accepted package exists
Main flow	1. Background transfer is scheduled under network constraints. 2. When constraints are satisfied the package is uploaded. 3. On success the local copy is marked transferred.
Alternative flows	No endpoint is configured, or transfer fails repeatedly (UC-6a): the operator is offered the manual share path.

A.3 Activity diagram

Figure 11 — Activity diagram for one capture session

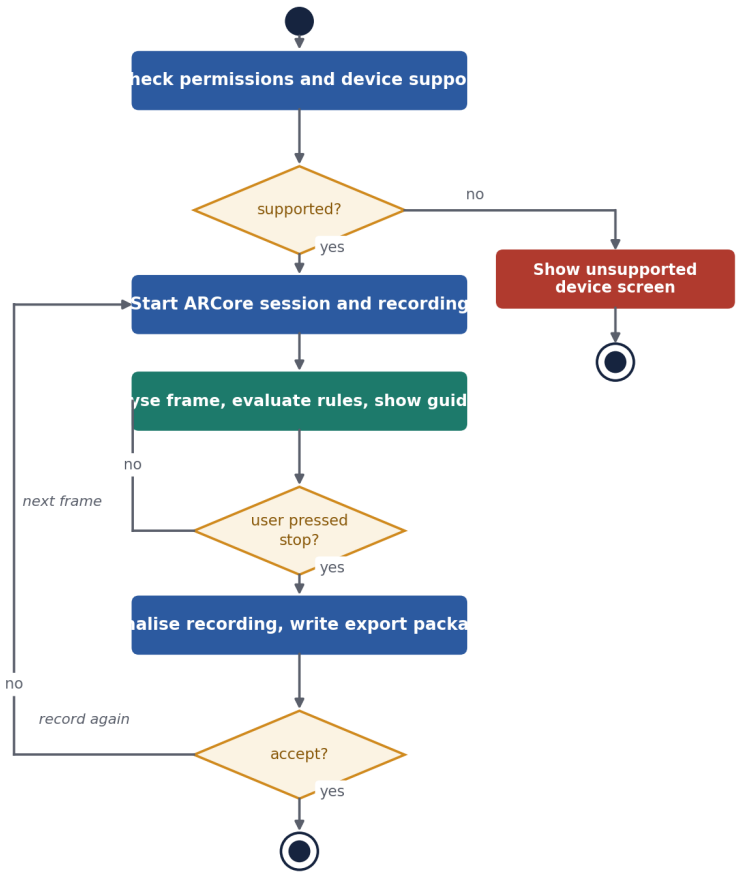


Figure 11 — Activity diagram for one capture session, including the unsupported-device path and both loops.

A.4 Class diagram

Figure 12 — Class diagram (core domain)

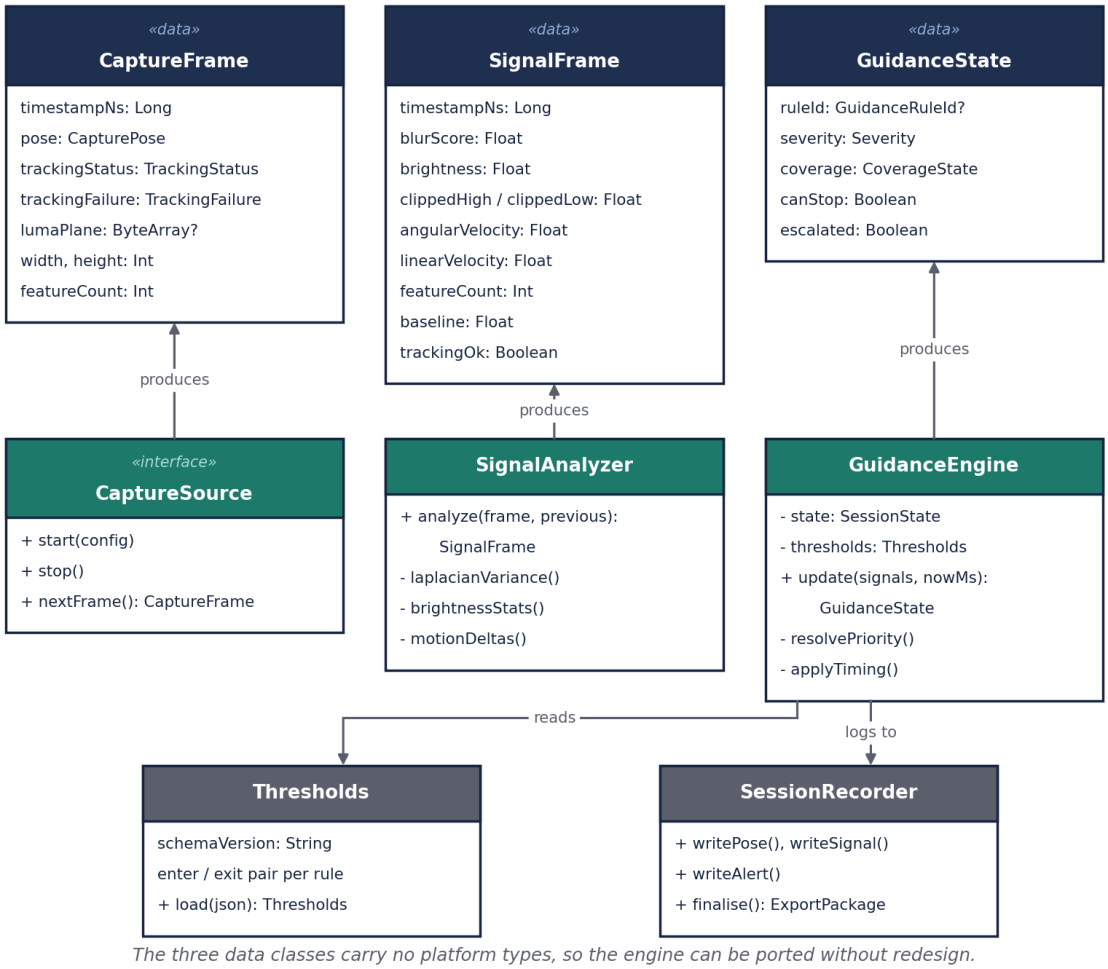


Figure 12 — Core domain classes. The three data classes carry no platform types, which is what allows the analysis and decision classes to be tested without a device.

CaptureSource is an interface with two implementations: one backed by ARCore for the application, and one backed by recorded sessions for the offline replay harness. The decision engine cannot distinguish between them, which is precisely the property that makes calibration possible.

A.5 Implementation responsibilities

Responsibility is allocated by module so that ownership is unambiguous and the modules can progress in parallel once the data contracts are fixed.

Area	Modules	Front end or back end	Principal work
AR capture and export	capture, export	Back end	ARCore session and recording, pose and point-cloud extraction, intrinsics, package writer, manifest, transfer
Signal analysis and interface	signals, react-native-app	Both	Image and motion measurement with unit tests; capture and summary screens, banner, pose trail, string resources
Decision engine and bridge	core-model, guidance, replay, bridge	Back end	Data contracts, rule set, priority, state machine, timing, offline replay harness, React Native module and AR view host

Cross-cutting responsibilities — the data contracts, the threshold configuration and the export format — are changed only by agreement, because a change to any of them affects more than one module and, in the case of the export format, affects Rysera as well.

End of document